

Department of Computer Science and Engineering, IIT Palakkad

CS 2030 Data Structures and Algorithms

Quiz 1 (27 Feb 2024)

Time: 8:00 hrs to 8:50 hrs

Points: 25

Name: ... *Knithika*

Roll No:

Instructions

1. This quiz-booklet has 8 pages containing 4 questions. Please get a new booklet if any question/page is missing.
2. Write your **name** and **roll number** in the space above.
3. Answer each question in the corresponding space provided. **Additional sheets may be used only for rough work and will not be evaluated.**
4. Write your answers neatly in blue or black ink. Do not use pencil or red ink.
5. Do not keep any books, papers, mobile phones or other smart gadgets with you during the exam.

1. Show that $n^2 - 5n$ is in $\Theta(n^2)$.

4

(1) $n^2 - 5n \leq n^2 \quad \forall n \geq 0$ 1 mark

(2) $\frac{1}{2}n^2 \leq n^2 - 5n \quad \forall n \geq 10$ 1 mark

$\therefore \frac{1}{2}n^2 \leq n^2 - 5n \leq n^2 \quad \forall n \geq 10$

$\therefore c_1 n^2 \leq n^2 - 5n \leq c_2 n^2 \quad \forall n \geq n_0$ 1 mark

where $c_1 = 1/2, c_2 = 1, n_0 = 10$

$\therefore$ By defn of $\Theta(n^2)$, $n^2 - 5n \in \Theta(n^2)$ 1 mark

(or) $\lim_{n \rightarrow \infty} \frac{n^2 - 5n}{n^2} = \lim_{n \rightarrow \infty} \frac{n^2(1 - 5/n)}{n^2(1)} = 1 \rightsquigarrow$ a constant $\neq 0$ 2 marks

$\therefore$ By defn of $\Theta(n^2)$, $n^2 - 5n \in \Theta(n^2)$ 1 mark

2. Explain how to implement two stacks in one array of size n in such a way that neither stack overflows unless the total number of elements in both stacks together is n . The PUSH and POP operations should run in $\mathcal{O}(1)$ time. Recall that a stack is said to overflow if a push operation is attempted when the stack is full.

6

Idea: Use left end of array for first stack + right end of array for second stack.

1 mark

Initialize $L.top = 0$, $R.top = n+1$

Push(S, x)

if $L.top + 1 = R.top$ } 1 mark
Return overflow

if $S = L$, $L.top++$ } (correct push)
else $R.top--$ } 1 mark
 $S[S.top] = x$

Pop(S)

if $S = L$ and $S.top = 0$ ret underflow 1 mark

if $S = R$ and $S.top = n+1$ ret underflow 1 mark

if $S = L$ }
 $L.top--$, return $L[L.top+1]$ } 1 mark
else
 $R.top++$, return $R[R.top-1]$

3. Let A be an array of size n such that $A[1] < A[2] < \dots < A[n]$. Design an $\mathcal{O}(\log n)$ -time algorithm to find an index i with $A[i] = i$ (if one exists). Prove correctness and analyze the running time of your algorithm.

Idea: $\text{mid} = \lfloor \frac{n}{2} \rfloor$

$A[\text{mid}] = \text{mid} \Rightarrow \text{return mid}$

$> \text{mid} \Rightarrow \text{Search in } A[1.. \text{mid}-1]$

$< \text{mid} \Rightarrow \text{Search in } A[\text{mid}+1..n]$

Algorithm Search (A) 2 marks

low = 1 high = n

while low $\leq$ high per iter const time

$\swarrow$
 $\log n$ times $\text{mid} = \lfloor (\text{low} + \text{high}) / 2 \rfloor$

if $A[\text{mid}] = \text{mid}$

return mid

else if $A[\text{mid}] < \text{mid}$

low = mid + 1

else

high = mid - 1

return -1

Recursive algo is also fine

Correctness 2 marks

$$A[mid] < mid \Rightarrow A[i] < i \quad \forall i \leq mid$$

$$A[1] \quad A[2] \quad \dots \quad A[mid-2] \quad A[mid-1] \quad A[mid]$$
$$< mid-2 \quad < mid-1 \quad < mid$$

$$A[mid] > mid \Rightarrow A[i] > i \quad \forall i \geq mid$$

$$A[1] \quad A[2] \quad \dots \quad A[mid] \quad A[mid+1] \quad A[mid+2]$$
$$> mid \quad > mid+1 \quad > mid+2$$

Running Time 1 mark

$$\text{Iter 1 : } low = 1 \quad high = n$$

$$\text{Iter 2 : } low = \left\lfloor \frac{n}{2} \right\rfloor + 1 \quad \text{or} \quad high = \left\lfloor \frac{n}{2} \right\rfloor - 1$$

In each iter, search space (ie) the positions where we search reduces by half

$$\begin{array}{lcl} \text{Iter 1 :} & n & \\ \text{Iter 2 :} & \frac{n}{2} & \\ \vdots & & \\ \text{Iter } k : & \frac{n}{2^k} = 1 & \therefore k = \lg n \end{array} \quad \left| \quad \begin{array}{l} n = 2^k \\ k = \lg n \end{array} \right.$$

If n is not a power of 2, ...

4. (a) Describe the Insertion Sort algorithm (with comments at appropriate places) to sort an array of n numbers.

3

Insertion Sort (x)

for $j = 2$ to n

$k = x[j]$

// Find correct position of k in $x[1..j-1]$

$i = j - 1$

while $i > 0$ + $x[i] > k$

$x[i+1] = x[i]$

$i = i - 1$

// $i = 0$ or $x[i] \leq k$

$x[i+1] = k$

- (b) State the loop invariant to argue that Insertion Sort is correct.

2

For $j = 2$ to $n+1$, at start of iter j of the for loop, $x[1..j-1]$ consists of elements originally in $x[1..j-1]$ but in sorted order

- (c) An inversion in an array $A[1..n]$ is a pair of indices (i, j) such that $i < j$ and $A[i] > A[j]$. The number of inversions in an n -element array is between 0 (if the array is sorted in non-decreasing order) and $\binom{n}{2}$ (if the array is sorted in non-increasing order). What is the relationship between the running time of Insertion Sort and the number of inversions in the input array? Justify your answer.

2

Consider $A[j]$ for $j = 2$ to n

No. of inversions of the form $i < j$

$$= \left| \{ i : i > 0, i < j, A[i] > A[j] \} \right|$$

(1 mark)

= No. of times the while loop is executed inside the for loop iter j

$\therefore$ Total no. of inversions = Total no. of times the while loop is executed

1 mark

- (d) Modify the Insertion Sort algorithm to count the number of inversions in an n -element array. Analyze the running time of your algorithm.

3

Count Inversions(x)

2 marks

$inv = 0$

for $j = 2$ to n

$k = x[j]$

// Find correct position of k in $x[1..j-1]$

$i = j-1$

while $i > 0$ + $x[i] > k$

$x[i+1] = x[i]$

$i = i-1, inv++$

// $i=0$ or $x[i] \leq k$

$x[i+1] = k$

Running Time of Algo is prop to the no. of inversions in the x/p arr A

$\therefore O(\text{no. of inversions})$ 1 mark

As no. of inversions $\leq \binom{n}{2}$,

running time is in $O(n^2)$